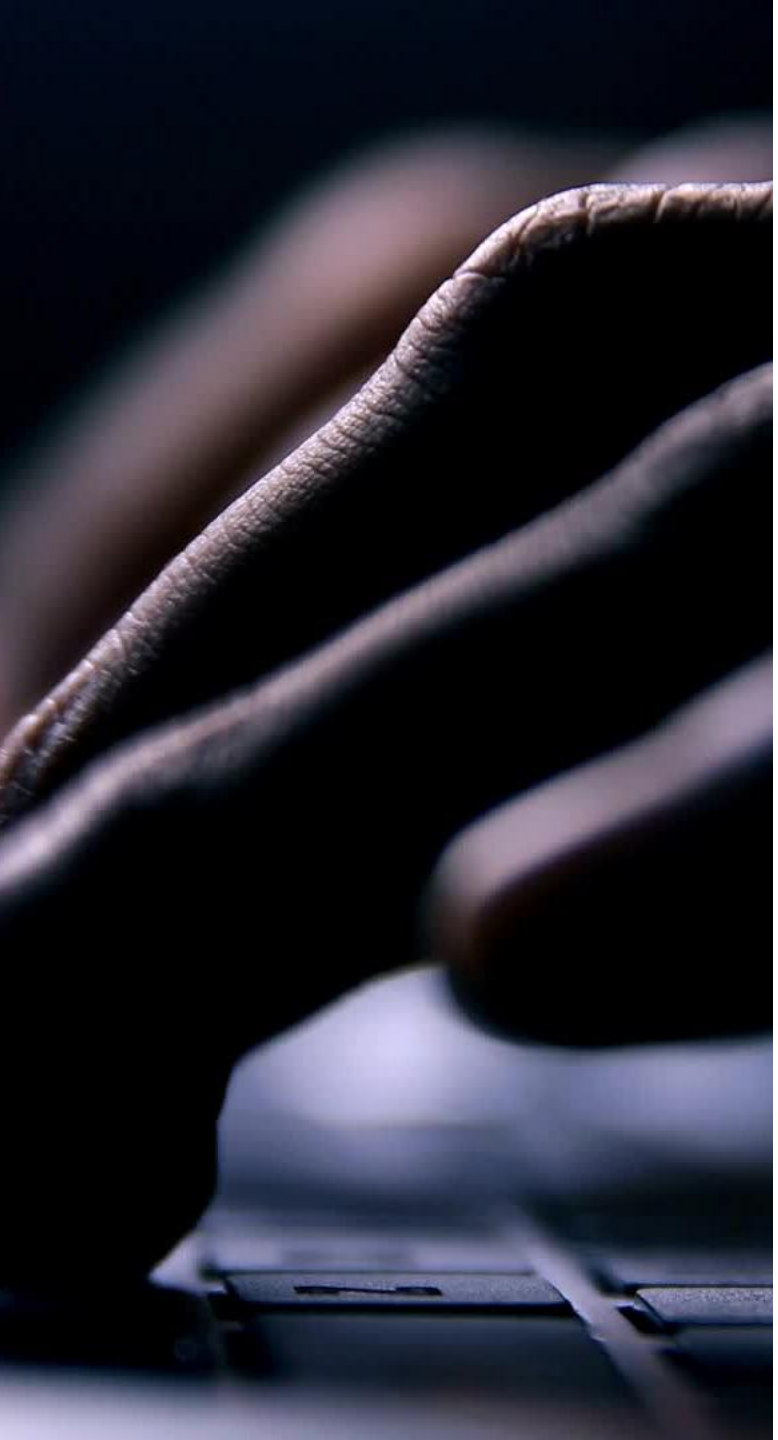


Fatec
Diadema
Luigi Papaiz

Programação para Dispositivos Móveis I

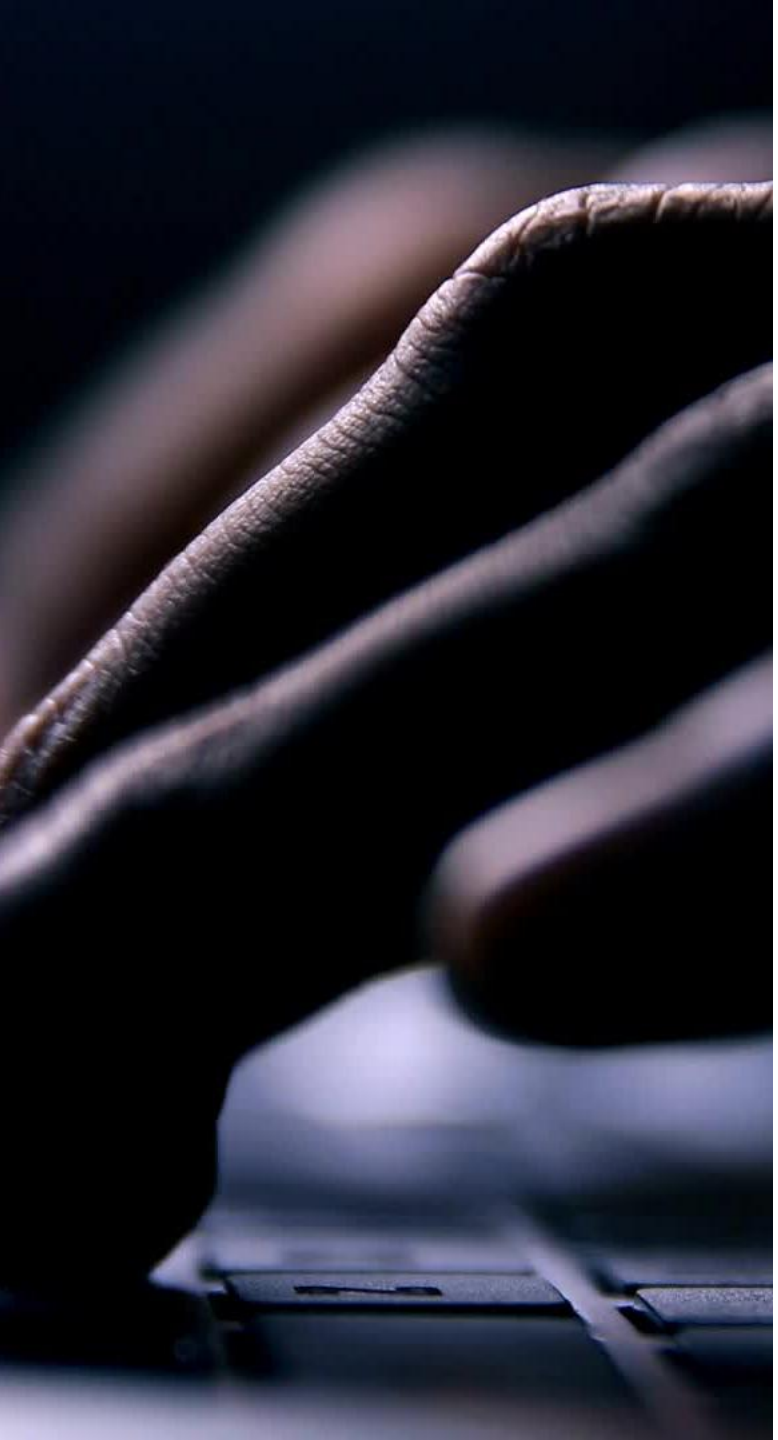
Prof. Rafael Miranda
rafael.miranda@cps.sp.gov.br

O problema

Vamos desenvolver uma calculadora em Kotlin.

Ela deverá realizar:

- Soma
- Subtração
- Multiplicação
- Divisão



Fatec
Diadema
Luigi Papaiz

Linguagem Kotlin - Funções

Primeira versão

Começamos sem funções:

```
val a = 10.0
val b = 5.0
val resultado = a + b


println(resultado)
```

Problema: e se precisarmos realizar várias operações?

Criando nossa primeira função


```
fun somar(a: Double, b: Double): Double {
    return a + b
}
```

Chamando:

```
val resultado = somar(10.0, 5.0)

println(resultado)
```

O que existe dentro da função?

Fatec
Diadema
Luigi Papaiz

```
fun somar(a: Double, b: Double): Double {
    return a + b
}
```

- fun - define uma função
- somar - nome
- a e b - parâmetros
- Double - tipo dos parâmetros
- Double - tipo do retorno
- return - devolve o resultado

Criando as outras operações

Fatec
Diadema
Luigi Papaiz

```
fun somar(a: Double, b: Double): Double {
    return a + b
}

fun subtrair(a: Double, b: Double): Double {
    return a - b
}

fun multiplicar(a: Double, b: Double): Double {
    return a * b
}

fun dividir(a: Double, b: Double): Double {
    return a / b
}
```

Agora temos quatro funções independentes

Expression Body

Observe:

```
fun somar(a: Double, b: Double): Double {
    return a + b
}
```

Como existe apenas uma expressão, podemos escrever:

```
fun somar(a: Double, b: Double) = a + b
```

Nossa calculadora


```
println(somar(10.0, 5.0))
println(subtrair(10.0, 5.0))
println(multiplicar(10.0, 5.0))
println(dividir(10.0, 5.0))
```

Como escolher qual operação executar?

Escolhendo a operação

Fatec
Diadema
Luigi Papaiz

```
val operacao = '+'


when (operacao) {
    '+' -> println(somar(10.0, 5.0))
    '-' -> println(subtrair(10.0, 5.0))
    '*' -> println(multiplicar(10.0, 5.0))
    '/' -> println(dividir(10.0, 5.0))
}
```

Será que podemos melhorar?

Nossa calculadora

Queremos esconder a lógica de escolha dentro de uma função:

```
fun calcular(
    a: Double,
    b: Double,
    operacao: Char
): Double {

    return when (operacao) {
        '+' -> a + b
        '-' -> a - b
        '*' -> a * b
        '/' -> a / b
        else -> 0.0
    }
}
```

Usando calcular()


```
println(calcular(10.0, 5.0, '+'))

println(calcular(10.0, 5.0, '-'))

println(calcular(10.0, 5.0, '*'))

println(calcular(10.0, 5.0, '/'))
```

Agora temos uma única função responsável pela calculadora.

Melhorando a chamada

Kotlin permite utilizar **argumentos nomeados**:

```
calcular(
    a = 10.0,
    b = 5.0,
    operacao = '*'
)
```

Isso pode deixar o código mais claro.

Parâmetro com valor padrão

Podemos definir uma operação padrão:

```
fun calcular(
    a: Double,
    b: Double,
    operacao: Char = '+'
): Double {
    ...
}
```

Agora:

```
calcular(10.0, 5.0)
```

Equivale a:

```
calcular(10.0, 5.0, '+')
```

E se quisermos somar vários números?

Nossa calculadora funciona muito bem para:

```
calcular(10.0, 20.0, '+')
```

Mas e se quisermos calcular:

10+20+30+40

Quantidade variável de argumentos

O Kotlin permite utilizar vararg.

```
fun calcular(vararg numeros: Double): Double {
    ...
}
```

vararg permite que a função receba **zero ou mais argumentos** daquele tipo.

vararg na função calcular()

Para começar, vamos implementar a soma:

```
fun calcular(operacao: Char, vararg numeros: Double): Double {

    return when (operacao) {

        '+' -> {
            var resultado = 0.0

            for (numero in numeros) {
                resultado += numero
            }

            resultado
        }

        else -> 0.0
    }
}
```

Utilizando a nova calcular()


```
val resultado = calcular(
    '+',
    10.0,
    20.0,
    30.0,
    40.0
)

println(resultado)
```

Uma função pode retornar mais de uma informação?

Até agora:

```
calcular(...)
```

retorna um único valor.

Mas imagine que quiséssemos retornar:

- resultado;
- quantidade de números utilizados;
- média.

Como retornar várias informações relacionadas?

data class

Podemos criar uma estrutura para representar o resultado:

```
data class Resultado(
    val resultado: Double,
    val quantidade: Int,
    val media: Double
)
```

Agora temos um único objeto contendo **várias informações**.

Retornando um Resultado


```
fun calcular(vararg numeros: Double): Resultado {

    var soma = 0.0

    for (numero in numeros) {
        soma += numero
    }

    val quantidade = numeros.size
    val media = soma / quantidade

    return Resultado(
        soma,
        quantidade,
        media
    )
}
```

Retornando um Resultado

Fatec
Diadema
Luigi Papaiz

```
val resultado = calcular(
    10.0,
    20.0,
    30.0
)

println(resultado.resultado)
println(resultado.quantidade)
println(resultado.media)
```

Atividade: Carrinho de compras

Fatec
Diadema
Luigi Papaiz

Uma loja precisa calcular informações sobre uma compra realizada por um cliente. A compra pode conter uma quantidade variável de produtos.

Crie um programa que deverá:

- receber os produtos da compra;
- calcular o valor total;
- retornar o valor total.
- a quantidade de produtos;
- o produto de maior valor.